Taming Sparse Rollout: Understanding Stability in RL for Large Language Models with Sparse Attention

Yang Zhou[1], Ranajoy Sadhukhan[1], Zhaofeng Sun[2], Zhuoming Chen[1], Souvik Kundu[3], Saket Dingliwal[4], Sai Muralidhar Jayanthi[4], Aram Galstyan[4], Haizhong Zheng[1], Beidi Chen[1]

[1]Carnegie Mellon University, [2]Independent Researcher, [3]Intel, [4]Amazon

Despite being powerful, reinforcement learning with verifiable rewards (RLVR) triggers extremely long COT, thus highly computationally expensive. RLVR per-step training cost is dominated by long-context generation in rollout, making sparse attention a promising technique to accelerate vanilla dense rollout generation. However, in practice, mastering the tradeoff between sparse rollout RL stability and efficiency is difficult - either sparsity too aggressive and training collapse or too lenient and insufficient speedup. To study the optimal tradeoff, we first observe that most tokens generated by sparse rollout are perfectly aligned with dense policy even under high sparsity. We then hypothesize that once we constrain the tail distribution of the sparse to dense actor-policy mismatch by a threshold, we are able to train with sparse rollout stably. We validate that our hypothesis holds by introducing a dynamic sparsity scheduling method for keeping the tail distribution constant through generation and study how the threshold scales with model size. Surprisingly, across a range of model sizes in Qwen3 thinking family, we find that keeping 5-percentile mismatch threshold above 0.86 generally works and use a cost-model analysis to find the a sparsity scheduling for maximum speedup under mismatch threshold, thus achieving **2.2x**, **2.4x**, and **2.0x** in rollout when training Qwen3-1.7B, Qwen3-4B, Qwen3-8B. Empirically, we show the identified threshold generalizes to much larger model size (Qwen3-14B) and other RL domain (Coding) and enables stable training. Additionally, we propose and show that DistillSparse, a lightweight LoRA-based distillation method that further actively aligns sparse rollouts with the dense policy and enables higher speedup with more aggressive sparsity.

Github: https://github.com/Infini-AI-Lab/TamedSparsity
Website: https://infini-ai-lab.github.io/tamed_sparsity_release/

1 Introduction

Reinforcement Learning with Verifiable Rewards (RLVR) has recently driven state-of-the-art performance in domains such as mathematical reasoning and code generation (OpenAI et al., 2026; Guo et al., 2025). However, these extended generations introduce substantial training inefficiency, since RL is increasingly bottlenecked by long-context inference during the rollout stage, which accounts for over 70% of per-iteration time (Wu et al., 2025b). Moreover, recent agentic tasks exacerbate this trend (Anthropic, 2026; Team et al., 2025; Su et al., 2025), where models are encouraged to produce trajectories that can extend beyond their nominal context window.

Dynamic sparse attention is a well-studied approach for accelerating long-context generation while preserving output quality (Tang et al., 2024a; Sun et al., 2024a; Yuan et al., 2025b). Thus, it is natural to use sparse attention to speed up rollouts for RL training of standard dense-attention policies. Yet despite this promise, there is limited adoption of sparse attention rollout in large-scale RL training of frontier dense models. **The key obstacle is a fundamental tradeoff between efficiency and stability**: aggressive sparsity can deliver larger speedups, but at the cost of a substantial distribution mismatch from the dense policy that can destabilize or even collapse RL training; milder sparsity is more stable, but offers only modest efficiency gains. This raises a practical question: **how to achieve the maximum speedup using sparse attention while maintaining dense-rollout stability across different model sizes.**

Prior work falls short of addressing this problem. (1) **Sparse Attention Inference Accuracy ↑ ≠ RL Stability ↑**: many sparse attention techniques have been developed to improve the efficiency–accuracy tradeoff

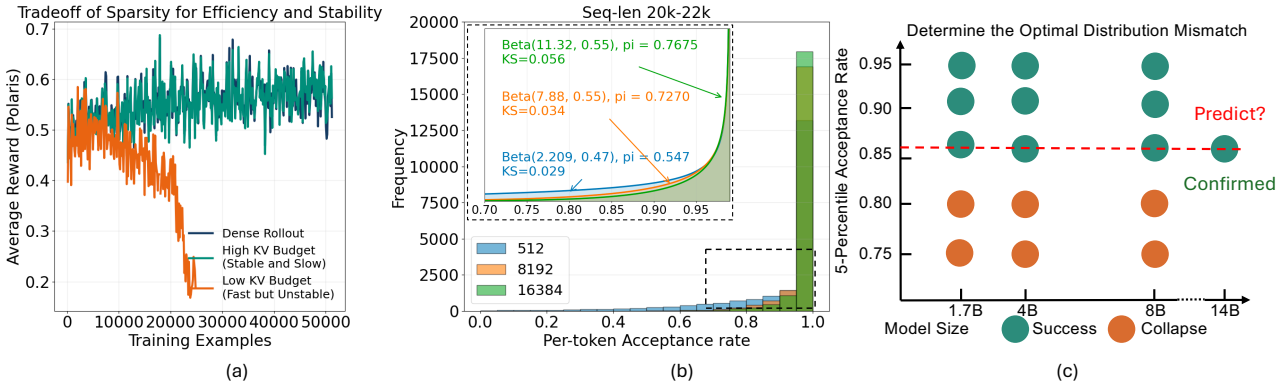

Figure 1 We study the tradeoff between training stability and efficiency using sparse rollout. (a) illustrates the main problem: arbitrary sparsity tends to miss the optimal tradeoff of lowest cost given stable training. (b) shows a case study demonstrating our insights, in the 20K–22K length bucket, regardless of lenient or aggressive sparsity, the per-token acceptance rate is long-tailed and heavily centered around 1. The high skewness of the distribution pushes its average acceptance rate toward one, making it less informative to evaluate the distribution. Instead, we choose 5-percentile per-token acceptance rate statistics to focus on the tail distribution as the basis of our study. Panel (c) displays the main problem: what is the lowest distribution alignment needed to enable stable dense-policy training. For thinking-model sizes 1.7B, 4B, and 8B, we train each model with sparse rollout targeting various levels of distribution mismatch, thus determining the optimal distribution mismatch.

for downstream-task inference (Tang et al., 2024b; Sun et al., 2024a; Xiao et al., 2024; Sadhukhan et al., 2025; Zhang et al., 2023). However, sparse rollout RL instability is mainly due to per-token distribution mismatch between sparse actor and dense policy rather than insufficient rollout rewards.* (2) **Suboptimal Convergence Under Severe Actor–Policy Mismatch**: prior work (Liu et al. (2025c), Liu et al. (2025b), Luo et al. (2026)) addresses actor–policy distribution mismatch, but often studies milder scenarios, such as staleness, where actor–policy KL divergence is one order of magnitude smaller than in sparse rollout. When directly applied, these techniques require clipping or masking significant training signals to maintain stability, leading to poor training convergence. (3) **Poor Efficiency**: Sparse rollout can be trivially recovered to achieve stable training by applying elementwise Top-k or using a huge KV budget, but these approaches do not achieve large efficiency gains.

Ideally, we desire RL with sparse rollout to (1) enable stable dense-policy training, (2) match dense performance regardless of model size and generation length, and (3) achieve strong efficiency benefits over dense rollout.

Our study is heavily inspired by the following insight. The key observation is that **sparse rollout collapse is not driven by a uniform degradation across all tokens**. Even under aggressive sparsity, most generated tokens remain nearly distribution-aligned with the dense policy. The unstable signal instead appears in the small fraction of tokens where sparse and dense behavior diverge. As shown in Figure 1(b), we collect tokens from sparse-attention generations with 20K prompt length and 2K generation length, then measure each token's distribution mismatch against the dense model. The resulting distribution is highly skewed: most tokens are close to perfectly aligned, even when the sparsity budget is aggressive enough to cause RL collapse. (In fact, in the paper we show that the tail distribution can be well-modeled by a Beta Distribution) **The high skewness makes average mismatch a weak stability indicator**. For example, when training Qwen3-1.7B with a 37K generation budget, a KV budget of 4096 remains stable while 2560 collapses, but their average per-token sparse-dense L1 distances are still very close and very sensitive to measurement precision: 0.977 and 0.968. We therefore evaluate sparse-dense mismatch with **lower-tail statistics** rather than the average. In particular, we use the lower 5-percentile per-token L1 distance to measure the worst-aligned tokens while ignoring the many "perfect" tokens that are unlikely to cause RL training collapse. From above formulation, we then hypothesize the following.

***Hypothesis**: if the tail distribution mismatch between the sparse actor and dense policy stays above a threshold throughout rollout, sparse rollout for dense policy RL will be stable.*

*In Appendix A we show that under sparse rollout that triggers dense policy collapse, even if we apply test-time-scaling and filtering to drastically increase the average rewards, even above those of dense rollout, the training performance is still not recovered.

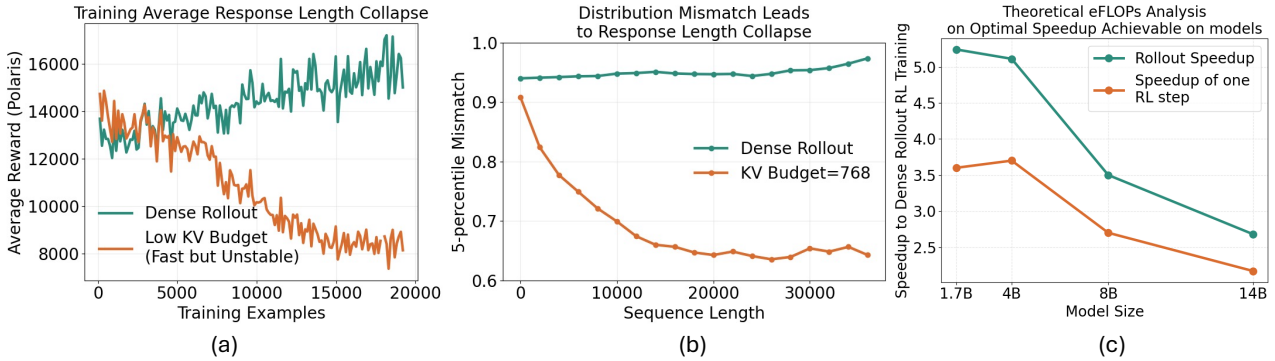

Figure 2 (a) Aggressive sparsity causes rollout response length to shrink during training, which correlates with dense-training collapse. (b) Unlike dense rollout, aggressive sparsity leads actor–policy distribution mismatch to grow sharply with sequence length. (c) Given we identified the optimal tail mismatch between sparse rollout and dense training is above 0.86 and under 0.90, we then estimate the theoretical optimal speedup ratio achievable from sparse rollout using our proposed cost model. Cost in (c) is estimated with eFLOPs on H200. We find that as the model size increases, the optimal speedup decreases. The two main reasons are first attention contribution to the total generation cost decreases as model sizes increases, and second we find that as model size increases, higher KV budget is required to maintain the actor-policy mismatch above the identified threshold, more details are provided in Section 3.3.

However, there are multiple challenges in verifying this hypothesis. First, keeping per-token sparse-dense distribution mismatch across the entire trajectory is not straightforward. As shown in Figure 2(b), under any fixed sparsity budget, the distribution mismatch always deteriorates as the generation length increases. Second, after holding mismatch statistics constant throughout the trajectory, it remains difficult to know which threshold for the mismatch to target and how it scales as model size increases. Third, even if the above hypothesis holds, as shown in Figure 1(c), how to achieve the mismatch target with minimal inference cost remains non-trivial.

In this paper, we systematically study sparse rollout and specifically answer the above questions:

- We introduce **sparsity scheduling**, which uses a more lenient sparsity budget as generation length increases to keep the per-token mismatch approximately constant throughout generation.
- In Section 3.2, we study the relationship between the target mismatch threshold and training stability. Surprisingly, we find that keeping the mismatch level above $\sim$0.86 leads to stable extended-step training across model sizes, supporting our hypothesis.
- In Section 3.3, we formulate RL rollout generation under dense and sparse cost models. Based on this cost model, we identify a practical scheduling guideline that achieves the best speedup under the stability constraint.
- In Section 4, We further demonstrate 2.2×, 2.4×, and 2.0× generation speedups for Qwen3-1.7B, Qwen3-4B, and Qwen3-8B.
- In Section 4.1 and 4.2, we then validate our hypothesis on Qwen3-14B thinking-model RL with a 37K generation cutoff, showing stable on par performance with dense rollout. Similarly, we also demonstrate that the dynamic sparsity schedule can generalizes to coding RL and achieve stable training on-par with dense rollout.
- In Section 5, we present an efficient LoRA distillation technique that can "break through" the threshold constraint and achieve higher speedups than plain sparse attention alone.

2 Problem: Sparse Attention Rollout Triggers RL Training Collapse

Due to limited space, comprehensive related work on sparse attention is presented in Appendix B. In our study, we focus on dynamic block-sparse attention as a case study. For inference tasks with long-context generation, dynamic sparsity has been extensively studied and shown to maintain inference accuracy while achieving significant speedups (Sadhukhan et al., 2025). However, in this section, we show that **choosing sparsity for sparse rollout in an ad hoc way poses a serious risk of training collapse**.

Figure 1(a) shows the result of training the Qwen3-1.7B Thinking model with a 40K context length on the PoLaris dataset (An et al., 2025) for math reasoning tasks. We apply TIS (Liu et al., 2025c) during RL training. [†] We present both standard dense-attention rollout and Block-Top-k sparse rollout with page size 16 and 64 pages (total KV budget 1024). With TIS, we follow the basic GRPO (Shao et al., 2024) training objective below. The TIS term is highlighted in red.

$$\mathcal{L}^{\text{PPO}}(\theta) = \mathbb{E}_{x\sim P_{\text{inf}}}\left[\min\left(\frac{p_{\text{ref}}(x)}{p_{\text{inf}}(x)}, C\right)\min\left(r_\theta(x)\hat{A}(x), \text{clip}(r_\theta(x), 1-\epsilon, 1+\epsilon)\hat{A}(x)\right)\right] \tag{1}$$

The sparse-rollout training reward curve completely collapses. We analyze the training instability from two perspectives: rollout generation length and actor–policy KL divergence.

First, as shown in Figure 2(a), the average rollout generation length for dense-policy training with dense rollout steadily increases, while the average rollout generation length for sparse rollout shrinks severely. We believe that the failure to scale up generation length is a fundamental cause of training instability. One main reason behind generation-length collapse is that the per-token distribution mismatch of sparse rollout grows as generation length increases. This corrupts the per-token training signal, makes it increasingly noisy, and ultimately causes the generation length to shrink. Second, although TIS has been effective in prior settings, the distribution mismatch here is significantly more severe than in scenarios where TIS succeeds. As visualized in Appendix Figure 10, the sparse–dense actor–policy mismatch is orders of magnitude larger than in traditional TIS-solvable settings such as actor staleness.

Admittedly, one can relax the sparsity setting and increase the KV budget to improve the chance of stable RL training by reducing the actor–policy distribution gap. However, how do the optimal settings vary across model sizes? More importantly, how can we select optimal sparse configurations without grid-search-style training over every possible configuration, especially for much larger models that are prohibitively expensive to train even once? **Therefore, to realistically unlock the efficiency benefits of sparse attention in long-context rollout, we need to systematically study the lowest sparse-rollout cost that still enables stable dense-model training, as outlined in the next sections.**

3 Observation and Insights

In this section, we present the insights and lay out the detailed study of the relationship between sparse attention and actor-policy distribution mismatch. Specifically, we show in 3.1 that even under aggressive sparsity, the majority of tokens exhibit perfect alignment with the dense model, while only a minor portion of tokens behave poorly. Based on the observation, in 3.2 we choose per-token metrics of distribution mismatch focusing on the tail and empirically show that the threshold for training stability is consistent over model sizes. Furthermore, 3.3 factors in cost considerations: the minimum sparse-attention cost needed to enable stable dense training.

3.1 Analysis on per-token Acceptance Rate: High Skewness Makes Average Acceptance Rate less Meaningful

Acceptance Rate α or L1 distance between two distributions is widely used (Leviathan et al., 2023) to measure the divergence between two distributions P and Q, which can be formulated as follows.

$$\alpha = \mathbb{E}_{x\sim Q}\left[\min\left(\frac{p(x)}{q(x)}, 1\right)\right] = \mathbb{E}_{x\in V}[\min(p(x), q(x))] \tag{2}$$

Naturally, given a sparsity configuration of constant KV budget, the acceptance rate decreases as the generation sequence length increases. Similarly, when looking at acceptance rate with a specific sequence length, the acceptance rate decreases as the sparsity configuration becomes more aggressive, or the KV budget becomes smaller. However, in this section, **we reveal that simply averaging the per-token acceptance rate given a sparsity configuration and sequence length is not ideal as an estimate of the distribution mismatch between the sparse and dense probability distributions**.

The core problem is that given a population of tokens collected under the same sparsity configuration and sequence length, the dominant the majority of tokens see their per-token acceptance rate above 0.999, perfectly aligned with

[†] If TIS is not applied, even dense-policy training with dense rollout will crash, since there is a mismatch in distribution between the SGLang rollout engine and the FSDP training forward pass, which are not bitwise aligned, as shown in (He and Lab, 2025). The 40K generation-length cutoff is extremely long and magnifies actor–policy divergence, causing crashes even in dense-to-dense training.

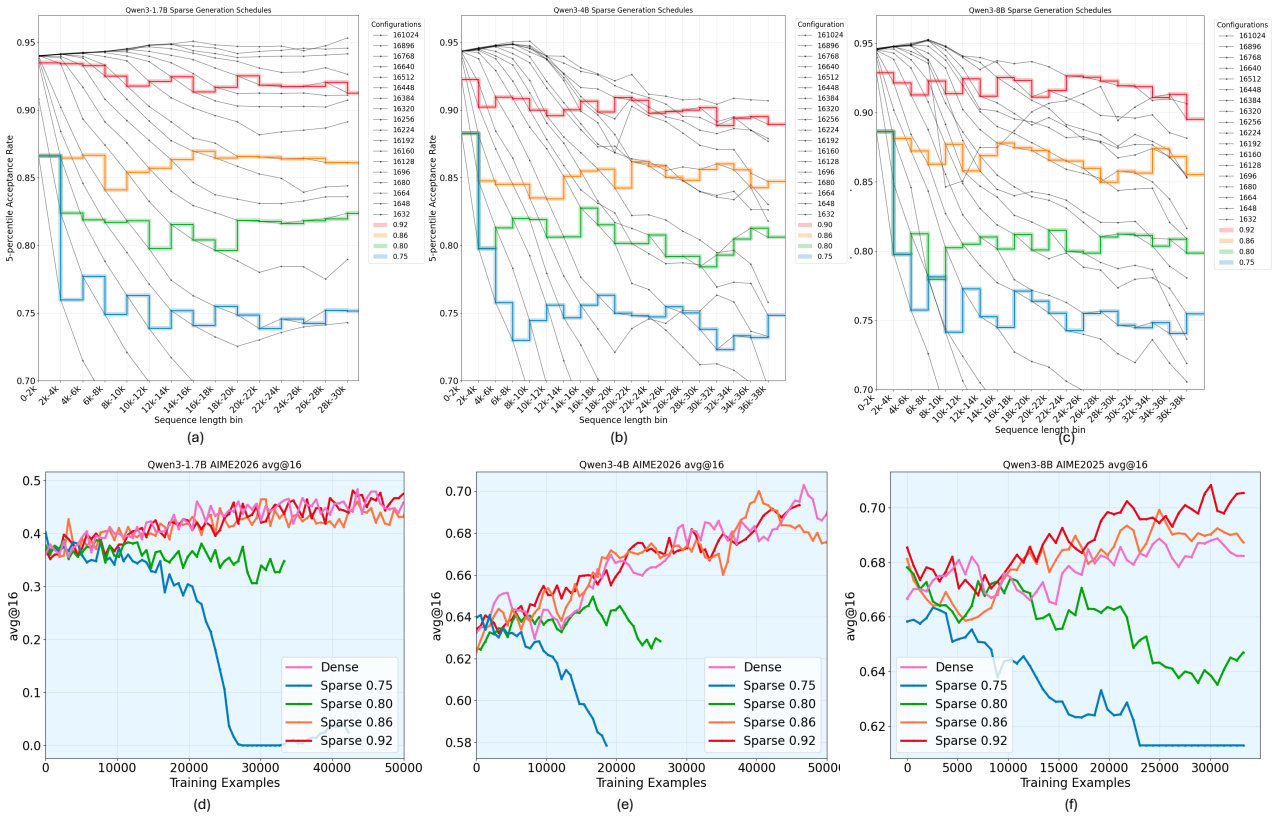

Figure 3 Control Study 1. Each column corresponds to one model: Qwen-1.7B, Qwen-4B, or Qwen-8B. The first row plots the 5-percentile acceptance rate versus sequence length for different sparsity configurations, all of which exhibit decaying acceptance rates as sequence length increases. We determine sparsity schedules that control per-trajectory divergence at 0.75, 0.8, 0.86, and 0.92. With these schedules, the second row evaluates whether each divergence target enables stable training with performance on par with dense rollout.

the dense policy. The high skewness towards 1 causes a severe problem that per-token acceptance rate is almost not correlated with sequence length increases, and the average acceptance rate falls into similar values regardless of aggressive and lenient sparsity. We use Qwen3-1.7B Thinking model with an aggressive KV budget (512) to generate sequences up to length 40K [‡] and compute each token's per-token acceptance rate. We calculate the Pearson's r statistic between per-token acceptance rate and sequence length, and the value is consistently around -0.1, suggesting a very weak correlation. Similarly, shown in Figure 1, we use KV budget of 512, 4096, and 16384 and look at all the tokens between 20k and 22k. Even though the KV budget differs significantly, the average acceptance rates in the 20k–22k sequence-length bucket are all pushed close to 1. **In practice, KV. budget of 2560 and 4096 gives average acceptance rate of 0.968 and 0.977, but one causes crash and one does not.**

Unlike the average acceptance rate, which is less informative when the distribution is highly skewed, statistics that focus on the tail distribution matter much more for evaluating the distribution mismatch between the approximate distribution and the original distribution. Notice that besides sharing similar average, the smaller KV budget distribution possesses a much thicker tail than higher KV budget. Specifically, we find that if we leave out all the tokens with per-token acceptance rate higher than 0.999, the tail can be modeled very nicely by a scaled Beta distribution of support [0, 0.999], with all three distributions having Kolmogorov-Smirnov (KS) statistic under 0.04. From the distribution for the run with KV budget 512, the variance of the respective beta distribution is 0.19, much more spread out than 0.05 for KV budget 16384.

Conceptually, tokens perfectly aligned with dense policy are not the culprit of training collapse. The issue of training collapse comes from the minority of tokens that are drastically different from dense in probability distribution. Therefore, instead of focusing on the average per-token acceptance rate, we focus on statistics

[‡]we ask the inference engine to also return per-token position Top 20 tokens and logprobs, which enables us to approximate per-token acceptance rate at every token position

that focus on the tail distribution.

***Method**: we chose 5-percentile cutoff of the per-token acceptance rate for our study to focus on the tail distribution between actor and policy mismatch in RL training.*

Also, the 5-percentile cutoff achieves -0.55 Pearson's r, strongly correlated to sequence length. Shown in Figure 4(a), the 5-percentile per-token acceptance rate allows the same sparsity configurations to show consistent mismatch values between dense policy regardless of training iterations under the baseline dense rollout scenarios, almost the same between iteration 1 and iteration 550. Even though the dense policy improves as the training continues, same sparsity budget the mismatch is only relative to the corresponding dense policy.

3.2 Control Study 1: What Threshold of Actor–Policy Mismatch Enables Stable RL Training, and How Does It Scale with Model Size?

The main challenge to cleanly study the actor-policy mismatch to RL training stability with sparse attention is that the mismatch naturally increases as sequence length increases when the sparsity budget is fixed before generation. We address it by introducing the sparsity scheduling.

***Method**: We use **Sparsity Scheduling** which increases the sparsity budget as the sequence length increases, thus preserving the tail distribution mismatch of the entire generation trajectory bin following a targeted value.*

As shown in Figure 3(a–c), we first measure the 5-percentile acceptance rate of 18 sparsity configurations for each model across 20 sequence-length bins. All configurations use page size 16 with varying numbers of pages. We consider four distribution-divergence targets: 0.75, 0.80, 0.86, and 0.92. To attain each target, we search horizontally across sparsity settings to find the desired configuration for each divergence target and sequence-length regime. Across all three sized models, we follow similar procedures to find the configs.

Specifically, to make sure that our measurement of the per-token acceptance rate statistics is generalizable to math training dataset in general, we use AIME 2024, 2025, 2026 together repeating 8 times as the prompt set and using sparse attention models for generation. The sparse generation with log-prob distribution are computed with dense log-probs for per-token acceptance rate at each trajectory positions.

Also, as the per-token acceptance rate generally lowers as sequence length increases, to facilitate analysis, we group tokens based on their position sequence length with 2K as the bin size. 37K generation length encompasses tokens separated into 19 bins. We compute the per-bin tail statistics for our study.

Then, following the four divergence levels, we ensure that the trajectories sampled by sparse rollout reaches a steady 5-percentile acceptance rate. We train dense using rollout of four targets to see whether they enable stable dense policy RL, as shown in Figure 3 (d), (e), (f).

***Takeaway1**: The optimal distribution mismatch threshold are roughly between 0.85 and 0.90, and consistent through increasing model sizes.*

The takeaway is that it keeps the tail distribution in the vincinity of the dense distribution while leaving some room for efficiency improvement.

3.3 Control Study 2: What Is the Lowest Cost Needed for Stable and Performant Dense-Model Training?

To make sure the cost analysis can transfer to different hardware easily, we follow (Sadhukhan et al., 2025) and model the cost of rollout based on model sizes and hardware memory bandwidth. For dense attention, we can formulate the compute, memory, and combined cost of repeated sampling N times in eFLOPs (equivalent FLOPS), given model parameters P, input prompt length in tokens L_{out}, output prompt length in tokens L_{out}, dimension of both Key and Value D, GQA ratio (greater than 1) in r, and 1 over the GPU SRAM memory bandwidth I

$$C_{comp} = 2 \cdot P \cdot N \cdot L_{out} + r \cdot (2 \cdot L_{in} + L_{out}) \cdot L_{out} \cdot N \cdot D$$
$$C_{mem} = 2 \cdot L_{in} \cdot L_{out} \cdot D + N \cdot L_{out}^2 \cdot D$$
$$C_{dense} = C_{comp} + I \cdot C_{mem}$$

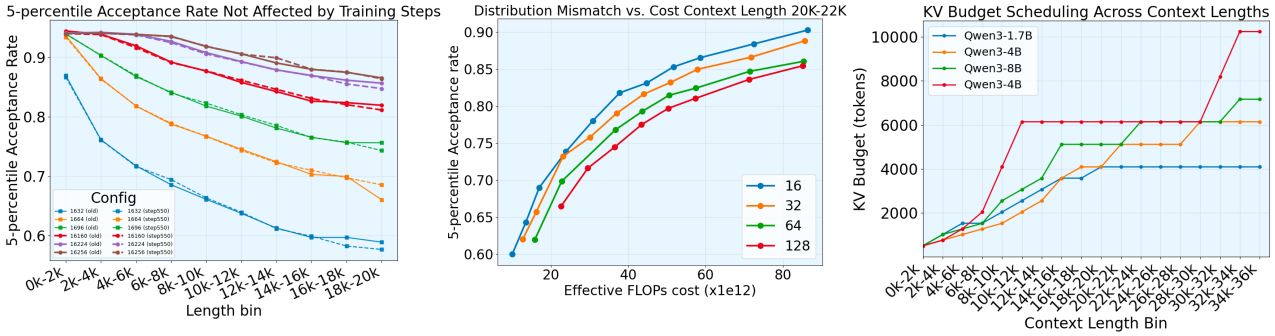

Figure 4 (a) shows an important property of the 5-percentile acceptance rate: it is not dependent on training iteration, and is instead determined by the dense model, sparsity level, and sequence length. (b) shows that page size 16 leads larger page sizes in the tradeoff between distribution alignment and efficiency. (c) shows that larger models generally favor larger KV budgets to maintain the same distribution-alignment level (0.86 here).

Similarly, we can model the same for block-sparse attention as follows. For simplicity, we assume that Top-k kernels incur only minimal overhead for $\texttt{pagesize} \geq 16$. Sparsity is with KV budget B and *pagesize*,

$$C_{\text{sparse, no scoring}} = \underbrace{2 \cdot N \cdot P \cdot L_{out} + 2 \cdot r \cdot N \cdot D \cdot B \cdot L_{out}}_{\text{compute}} + \underbrace{2 \cdot I \cdot N \cdot D \cdot B \cdot L_{out}}_{\text{memory}}.$$

$$C_{scoring} = \underbrace{2 \cdot N \cdot L_{in} \cdot D \cdot L_{out} + \frac{r \cdot N \cdot D \cdot L_{out}^2}{2 \cdot \text{pagesize}}}_{\text{compute}} + \underbrace{2 \cdot I \cdot L_{in} \cdot D \cdot L_{out} + \frac{I \cdot N \cdot D \cdot L_{out}^2}{2 \cdot \text{pagesize}}}_{\text{memory}}.$$

$$C_{sparse} = C_{\text{sparse, no scoring}} + C_{scoring}$$

Following the above cost-model formulation, we search over page sizes 16, 32, 64, and 128 for each model size: 1.7B, 4B, and 8B.[§] Based on the proposed cost model, we identify an important property of block-sparse attention: for page sizes greater than or equal to 16, page size 16 dominates the Pareto frontier for the tradeoff between tail acceptance rate and cost. Figure 4(b) shows this scenario for the 1.7B model at sequence lengths 20k–22k. The pattern holds true across other length regimes as well, more details in Appendix C. The core reason behind such phenomenon is that from page size 16 and onwards, the cost of computing landmark scores and picking Top-k is not expensive and much insignificant than the actual attention computation, as predicted by the cost model.

Takeaway 2: *For block-sparse attention ($\texttt{pagesize} \geq 16$), more fine-grained sparsity ($\texttt{pagesize}\downarrow$) consistently beats the coarser sparsity ($\texttt{pagesize}\uparrow$) in distribution mismatch with the dense model across all cost regimes.*

Importantly, beyond the pattern predicted on paper by the cost model, we show in Appendix C Table 4 that if the Top-k kernels are implemented properly, the empirical cost of sparse attention model generation is insensitive to page-size selection. Therefore, we know that to achieve the target acceptance rate with the smallest cost, we stick to using the uniform page size of 16 for sparsity going forward.

4 Empirical Studies

Following the plan laid out in 3.2, we train Qwen3 (Yang et al., 2025) family models 1.7B, 4B, 8B for one epoch on Polaris (An et al., 2025) using the planned sparsity schedule based on the tail acceptance rate statistics (5-percentile acceptance rate). Specifically, to enable high infrastructure utilization during training, we implement the dynamic sparsity scheduling generation required by our design based on open-sourced sparse attention library Vortex (Infini-AI-Lab, 2026). The results are summarized in Table 1 with tests of three different years 2024, 2025, 2026 of AIME problems, each averaged over 16 times, and two years 2024, 2023 of AMC problems each averaged over 4 times, showing that they are on par with dense.

[§]page size 8 or below requires much more system optimization and is currently not supported by our sparse inference engine. Future work will extend this analysis to smaller page sizes.

Table 1 Comprehensive Evaluation of Sparse Rollout relative to Dense, where we report the best-achieved scores under each downstream task and task-related metrics.

Models	AIME25 Mean@16	AIME25 Pass@16	AIME26 Mean@16	AIME26 Pass@16	AIME24 Mean@16	AIME24 Pass@16	AMC22/23 Mean@4	AMC12 Mean@4
Qwen3-1.7B Thinking 37K generation length cutoff; one epoch on 53K Polaris examples;								
Dense Rollout Dense Training	0.4322	0.5440	0.4833	0.5995	0.5687	0.7172	0.8072	0.7333
Sparsity setup (0.92) Rollout	0.4625	0.5729	0.4875	0.6197	0.5770	0.7155	0.834	0.7222
Sparsity setup (0.86) Rollout	0.4145	0.5436	0.4645	0.5914	0.5416	0.6958	0.8192	0.7166
Sparse with LoRA Distillation Sec. 5	0.4354	0.5562	0.4562	0.5874	0.5333	0.6929	0.7333	0.8072
Qwen3-4B Thinking 37K generation length cutoff; one epoch on 53K Polaris examples;								
Dense Rollout Dense Training	0.7145	0.7933	0.7229	0.8216	0.775	0.836	0.8	0.9367
Sparsity setup (0.92) Rollout	0.7	0.7978	0.7062	0.8158	0.7729	0.836	0.7944	0.9337
Sparsity setup (0.86) Rollout	0.7416	0.8321	0.7125	0.8197	0.756	0.832	0.7944	0.9397
Qwen3-8B Thinking 37K generation length cutoff; one epoch on 33K Polaris examples;								
Dense Rollout Dense Training	0.7083	0.8000	0.6979	0.8176	0.7770	0.8637	0.8111	0.9337
Sparsity setup (0.92) Rollout	0.7229	0.8076	0.725	0.8233	0.7815	0.8456	0.8111	0.927
Sparsity setup (0.86) Rollout	0.7145	0.8055	0.6979	0.8125	0.7791	0.8441	0.8277	0.9216
Qwen3-14B Thinking 37K generation length cutoff; one epoch on 53K Polaris examples;								
Dense Rollout Dense Training	0.75	0.8269	0.7770	0.8646	0.8125	0.8810	0.8055	0.9396
Sparsity setup (0.86) Rollout	0.75	0.828	0.7708	0.8558	0.8028	0.8696	0.8111	0.9367

Implementation Efficiency - To mimic the distribution of rollout during RL training of Polaris, we gather AIME24 and repeat every problem 8 times with 32K generation length cutoff. Results are shown in Table 2. For profiling, we deploy 4xH200 GPUs. We show that across 1.7B, 4B, and 8B, we can achieve over 2.0x speedup on RL rollout, which is around 2.0x speedup on the end-to-end iteration time.

Table 2 Throughput and speedup from dynamic sparsity scheduling, benchmarked on 4×H200 GPUs.

Model	Dense t.↑ (tokens/s)	Sparse t. ↑ (tokens/s)	Rollout Speedup	RL One-step Speedup
Qwen3-1.7B (0.86)	2561	5662	2.21×	1.97×
Qwen3-4B (0.86)	1369	3308	2.41×	2.11×
Qwen3-8B (0.86)	1300	2682	2.06×	1.81×
Qwen3-14B (0.86)	1544	2289	1.49×	1.35×

4.1 Testing Threshold on Much Larger Model

Beyond the three model sizes for which we have enough compute to sweep sparsity settings, we explore ways to extend our study to regimes where enumeration is no longer feasible. A Qwen3-14B Thinking model with a full 37K generation cutoff takes 4 H200 nodes (32 H200s) and 8 full days (190 hrs) to train for an entire epoch. For us, it is no longer possible to brute-force all potential options for the optimal sparsity schedule that maximize the efficiency benefit while preserving training convergence.

Table 3 Coding RL evaluation for Qwen3-1.7B Thinking with a 24K generation length cutoff after one epoch on 21K filtered TACO.

Setups	LiveCodeBench	HumanEval+	MBPP+
Dense Rollout Baseline	0.3970	0.6219	0.5846
Sparsity (0.86)	0.3933	0.6280	0.6084

We follow the 5-percentile acceptance rate guidance to keep 0.86 and train Qwen3-14B using the determined sparsity schedule. The training runs for the epoch on Polaris. The dense policy from sparse rollout matches the dense policy from dense rollout RL very closely. We show the average training reward figure in Figure 6(a), and we show that the average training reward is highly close to dense reward with gap consistently less than 1%. We also populate the downstream evals in Table 1, showing that the Qwen3-14B matches the dense rollout training convergence.

4.2 Testing Threshold Generalization on Coding RL

Beyond math reasoning, training strong coding model also requires RL to scale up CoT to achieve higher pass-rate in harder coding problems. We show that the threshold found using math domain can be directly used for coding training. For the experimental implementation, we follow Liu and Zhang (2025) to set up a sandbox for reward computation and use the 21K filtered TACO training examples from Li et al. (2023a) to train a Qwen3-1.7B Thinking model with the 0.86 sparsity setup. We evaluate the training performance with LiveCodeBench (Han et al., 2024), HumanEval+, and MBPP+ (Liu et al., 2023).

The results are reported in Table 3, where we show that the threshold determined in math reasoning generalizes to other generation workloads, coding. We also present RL training convergence in Figure 6(b), showing that sparse rollout matches dense-rollout convergence over extended training.

4.3 Optimal KV Budget Scales up as Model Sizes Increases

An important phenomenon we discovered is that as model size increases, the KV budget needed for generation at any sequence length also increases in order to hold the tail acceptance rate constant. In Figure 4, we show that across sequence length, 14B (red) requires the most KV budget to attain 5-percentile acceptance rate 0.86, followed by 8B (green), 4B (yellow), and 1.7B (blue). This result also contributes to the decreasing trend of optimal speedup achievable by these models shown in Figure 1(c) and explains the speedup shown for 14B in Table 2 using sparse attention.

5 Opportunities to Get More Speedup Beyond the Threshold

5.1 Case Study: DistillSparse

Although threshold studies enable us to understand the actor-policy divergence and acquire the best tradeoff of stable training and optimal efficiency, it is still important to explore opportunities to go beyond the optimal efficiency from the threshold for higher ceiling of efficiency improvement. Among potential techniques, we argue that one straightforward opportunity presents to actively bring the sparse rollout closer to dense model through lightweight distillation. Sparse rollout already generates extended trajectories, while dense policy training already requires computation of dense log-probabilities. Without additional significant computation cost, we can reuse the two to actively fine-tuned the sparse rollout directly with on-policy distillation.

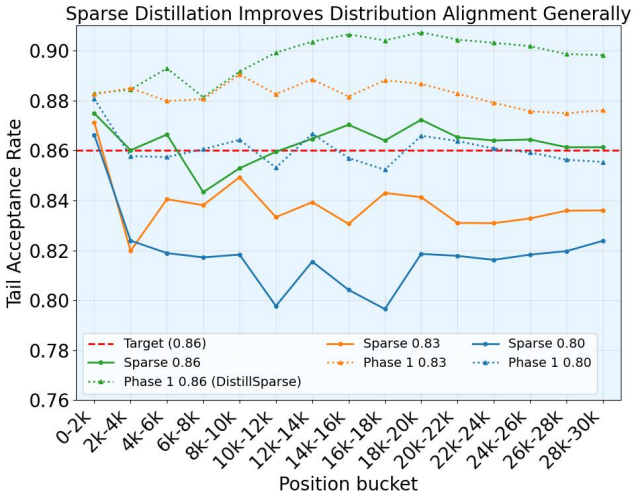

Figure 5 DistillSparse training on sparsity 0.86 after 20K training examples. We show that DistillSparse improves sparse attention to dense distribution generally, as the distribution alignment of sparsity 0.80 and 0.83. In fact, old sparsity with 0.80 tail mismatch now get boosted to 0.86 close to the identified threshold.

Then, we advocate for a LoRA-based distillation called DistillSparse that goes beyond previous limit. The complete description of each step of RL training is described in Algorithm 1. The on-policy distillation process is naturally viable: no the extra computation required for new sparse student trajectory generation, and the dense attention teacher log-probs used to compute dense policy update (step 4 in Algorithm 1) can be repurposed to compute the distillation loss in step 7. The core challenge is to distill the sparse rollout without contaminating the training signal of the dense policy weights. Thus, we propose training a set of LoRA weights for aligning sparse and that are added only during rollout (sparse generation), discarded during dense policy log-probability computation and dense policy training. **Next, we illustrate that with distillation of DistillSparse, sparse attention is more aligned to dense in general and enables much more aggressive sparsity to attain the same distribution mismatch threshold we found to enable stable RL training with higher speedup.**

Algorithm 1 Online LoRA distillation for sparse RL training

Require: Base model parameters θ; LoRA parameters ϕ; prompt batch $x \sim \mathcal{D}$

Construct **dense policy** $\pi_\theta^{\text{dense}}$ (full attention) and **sparse policy** $\pi_{\theta,\phi}^{\text{sparse}}$ (sparse attention + LoRA)
for each RL iteration **do**
  Sample rollouts $y \sim \pi_{\theta,\phi}^{\text{sparse}}(\cdot \mid x)$ and compute rewards / advantages
  Recompute token log-probs with the dense model (FSDP): $\log \pi_\theta^{\text{dense}}(y_t \mid x, y_{<t})$
  Compute token log-probs with the sparse model: $\log \pi_{\theta,\phi}^{\text{sparse}}(y_t \mid x, y_{<t})$
  Update the base model $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{PPO}}\left(\theta; x, y, \log \pi_\theta^{\text{dense}}\right)$
  Freeze θ and update LoRA $\phi \leftarrow \phi - \eta_\phi \nabla_\phi \mathcal{L}_{\text{LoRA}}\left(\phi; x, y, \theta\right)$
  Use LoRA loss $\mathcal{L}_{\text{LoRA}} = \mathcal{L}_{\text{PPO\_Sparse}} + \lambda \mathcal{L}_{\text{KL}}$
  Refresh inference engine weights with updated base model θ and LoRA ϕ
end for

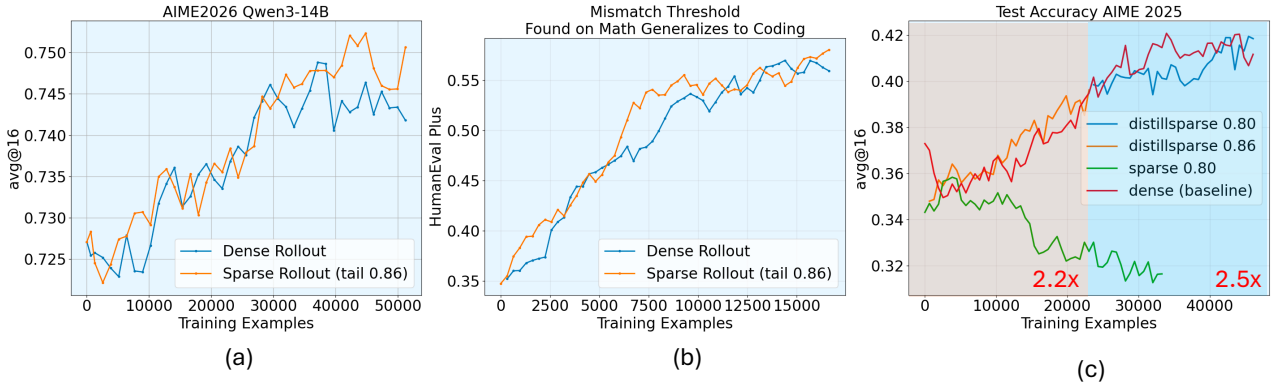

Figure 6 (a) We show the training stability of the Qwen3-14B Thinking model. Under a 37K generation length cutoff on Polaris, rollout with the 0.86 sparsity setting achieves downstream results on par with dense rollout. (b) Sparse rollout with the identified threshold also enables stable RL training beyond math data, matching dense-rollout performance on coding. (c) DistillSparse provides an opportunity to maintain training stability while gaining additional speedup.

5.2 How DistillSparse Breaks the Vanilla Sparse Rollout Efficiency Threshold

For our proof-of-concept case study, we run the experiment with Qwen3-1.7B Thinking model. With the goal of achieving training stability, we start with the 0.86 dynamic sparsity schedule but with LoRA activated. For every iteration trained, both the dense policy model and the sparse LoRA are updated, which further boosts the distribution alignment of the 0.86 sparsity (speedup 2.2x) with dense.

Shown in Figure 5, after training for 20k training examples (phase 1), the green solid and dashed curves show the effect of added LoRA weights and on-policy distillation on the sparse rollout on the original 0.86 sparsity setting. We see that through distillation, the same sparsity level achieves significantly higher distribution alignment across all generation sequence lengths. (green dashed higher than green solid line) More importantly, we show that DistillSparse generally boosts the 5-percentile acceptance rate across other sparse-attention settings: the original 0.83 setting (coral, speedup 2.3x) and 0.80 setting (blue, speedup 2.5x) also achieve higher distribution alignment after distillation. Surprisingly, the original sparsity level of 0.80 now achieves a tail acceptance rate around the 0.86 threshold, and we switch to sparsity of 0.80 from now forward. We show in Figure 6 (c) that with the switching, the training is still stable and on par with dense for an extended number of steps. More importantly, for steps after 20K training examples, we can train for 2.5x speedup in the rollout, higher than in the first 20K training examples. We also show full empirical validation that DistillSparse matches dense rollout in Table 1.

Low Overhead - Moreover, DistillSparse can be readily integrated into our existing dynamic sparsity scheduling implementation with minimal overhead on top of original sparse rollout speedup. Details are presented in Appendix D.

6 Conclusion

Sparse attention can substantially reduce rollout cost in long-chain-of-thought RL, but naive sparse rollout introduces actor–policy mismatch severe enough to destabilize training. In this work, we show that this instability is governed less by average distribution shift than by the tail behavior of a small subset of misaligned tokens, which motivates using lower-percentile acceptance-rate statistics to characterize the true training-relevant divergence. Based on this view, we derive practical sparsity schedules that preserve stable dense-policy training across model scales while still delivering meaningful rollout speedups. We further show that DistillSparse improves sparse–dense alignment with minimal overhead, enabling even more aggressive sparse rollout settings to remain trainable. Together, these results provide a practical recipe for making sparse rollout both efficient and stable in RL for large language models.

References

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms. *arXiv preprint arXiv:2402.14740*, 2024.

Chenxin An, Zhihui Xie, Xiaonan Li, Lei Li, Jun Zhang, Shansan Gong, Ming Zhong, Jingjing Xu, Xipeng Qiu, Mingxuan Wang, and Lingpeng Kong. Polaris: A post-training recipe for scaling reinforcement learning on advanced reasoning models, 2025. https://hkunlp.github.io/blog/2025/Polaris.

Anthropic. Claude code. https://code.claude.com/, 2026. AI coding assistant.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling, 2023. https://arxiv.org/abs/2302.01318.

Zhuoming Chen. Vortex documentation, 2025. https://infini-ai-lab.github.io/vortex_torch/.

DeepSeek-AI. Deepseek-v3.2-exp: Boosting long-context efficiency with deepseek sparse attention, 2025.

Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures, 2018. https://arxiv.org/abs/1802.01561.

Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, et al. Areal: A large-scale asynchronous reinforcement learning system for language reasoning. *arXiv preprint arXiv:2505.24298*, 2025.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

K Han, A Gu, WD Li, F Yan, T Zhang, S Wang, A Solar-Lezama, K Sen, and I Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

Horace He and Thinking Machines Lab. Defeating nondeterminism in llm inference. *Thinking Machines Lab: Connectionism*, 2025. doi: 10.64434/tml.20250910. https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/.

Jian Hu, Xibin Wu, Zilin Zhu, Xianyu, Weixun Wang, Dehao Zhang, and Yu Cao. Openrlhf: An easy-to-use, scalable and high-performance rlhf framework. *arXiv preprint arXiv:2405.11143*, 2024.

Wei Huang, Yi Ge, Shuai Yang, Yicheng Xiao, Huizi Mao, Yujun Lin, Hanrong Ye, Sifei Liu, Ka Chun Cheung, Hongxu Yin, Yao Lu, Xiaojuan Qi, Song Han, and Yukang Chen. Qerl: Beyond efficiency – quantization-enhanced reinforcement learning for llms, 2025. https://arxiv.org/abs/2510.11696.

Infini-AI-Lab. Vortex: A flexible and efficient sparse attention framework, 2026. https://github.com/Infini-AI-Lab/vortex_torch. GitHub repository.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding, 2023. https://arxiv.org/abs/2211.17192.

Rongao Li, Jie Fu, Bo-Wen Zhang, Tao Huang, Zhihong Sun, Chen Lyu, Guang Liu, Zhi Jin, and Ge Li. Taco: Topics in algorithmic code generation dataset. *arXiv preprint arXiv:2312.14852*, 2023a.

Ziniu Li, Tian Xu, Yushun Zhang, Zhihang Lin, Yang Yu, Ruoyu Sun, and Zhi-Quan Luo. Remax: A simple, effective, and efficient reinforcement learning method for aligning large language models. *arXiv preprint arXiv:2310.10505*, 2023b.

Guangda Liu, Chengwei Li, Jieru Zhao, Chenqi Zhang, and Minyi Guo. Clusterkv: Manipulating llm kv cache in semantic space for recallable compression, 2025a. https://arxiv.org/abs/2412.03213.

Hongyi Liu, Zhuoming Chen, Yang Zhou, Haizhong Zheng, and Beidi Chen. Jackpot: Optimal budgeted rejection sampling for extreme actor-policy mismatch rl. https://github.com/Infini-AI-Lab/jackpot.git, 2025b. Official GitHub repository.

Jiawei Liu and Lingming Zhang. Code-r1: Reproducing r1 for code with reliable rewards. 2025.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatGPT really correct? rigorous evaluation of large language models for code generation. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. https://openreview.net/forum?id=1qvx610Cu7.

Liyuan Liu, Feng Yao, Dinghuai Zhang, Chengyu Dong, Jingbo Shang, and Jianfeng Gao. Flashrl: 8bit rollouts, full power rl, August 2025c. https://fengyao.notion.site/flash-rl.

Xiaoxuan Liu, Jongseok Park, Langxiang Hu, Woosuk Kwon, Zhuohan Li, Chen Zhang, Kuntai Du, Xiangxi Mo, Kaichao You, Alvin Cheung, Zhijie Deng, Ion Stoica, and Hao Zhang. Turbospec: Closed-loop speculation control system for optimizing llm serving goodput, 2025d. https://arxiv.org/abs/2406.14066.

Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl. https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2, 2025. Notion Blog.

Sijia Luo, Xiaokang Zhang, Yuxuan Hu, Bohan Zhang, Ke Wang, Jinbo Su, Mengshu Sun, Lei Liang, and Jing Zhang. Sparse-rl: Breaking the memory wall in llm reinforcement learning via stable sparse rollouts. *arXiv preprint arXiv:2601.10079*, 2026.

Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, Benjamin Bossan, and Marian Tietz. PEFT: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft, 2022.

Yu Meng, Mengzhou Xia, and Danqi Chen. Simpo: Simple preference optimization with a reference-free reward. *Advances in Neural Information Processing Systems*, 37:124198–124235, 2024.

OpenAI, :, Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, Alex Iftimie, Alex Karpenko, Alex Tachard Passos, Alexander Neitz, Alexander Prokofiev, Alexander Wei, Allison Tam, Ally Bennett, Ananya Kumar, Andre Saraiva, Andrea Vallone, Andrew Duberstein, Andrew Kondrich, Andrey Mishchenko, Andy Applebaum, Angela Jiang, Ashvin Nair, Barret Zoph, Behrooz Ghorbani, Bohan Zhang, Ben Rossen, Benjamin Sokolowsky, Boaz Barak, Bob McGrew, Borys Minaiev, Botao Hao, Bowen Baker, Brandon Houghton, Brandon McKinzie, Brydon Eastman, Camillo Lugaresi, Cary Bassin, Cary Hudson, Chak Ming Li, Charles de Bourcy, Chelsea Voss, Chen Shen, Chong Zhang, Chris Koch, Chris Orsinger, Christopher Hesse, Claudia Fischer, Clive Chan, Dan Roberts, Daniel Kappler, Daniel Levy, Daniel Selsam, David Dohan, David Farhi, David Mely, David Robinson, Dimitris Tsipras, Doug Li, Dragos Oprica, Eben Freeman, Eddie Zhang, Edmund Wong, Elizabeth Proehl, Enoch Cheung, Eric Mitchell, Eric Wallace, Erik Ritter, Evan Mays, Fan Wang, Felipe Petroski Such, Filippo Raso, Florencia Leoni, Foivos Tsimpourlas, Francis Song, Fred von Lohmann, Freddie Sulit, Geoff Salmon, Giambattista Parascandolo, Gildas Chabot, Grace Zhao, Greg Brockman, Guillaume Leclerc, Hadi Salman, Haiming Bao, Hao Sheng, Hart Andrin, Hessam Bagherinezhad, Hongyu Ren, Hunter Lightman, Hyung Won Chung, Ian Kivlichan, Ian O'Connell, Ian Osband, Ignasi Clavera Gilaberte, Ilge Akkaya, Ilya Kostrikov, Ilya Sutskever, Irina Kofman, Jakub Pachocki, James Lennon, Jason Wei, Jean Harb, Jerry Twore, Jiacheng Feng, Jiahui Yu, Jiayi Weng, Jie Tang, Jieqi Yu, Joaquin Quiñonero Candela, Joe Palermo, Joel Parish, Johannes Heidecke, John Hallman, John Rizzo, Jonathan Gordon, Jonathan Uesato, Jonathan Ward, Joost Huizinga, Julie Wang, Kai Chen, Kai Xiao, Karan Singhal, Karina Nguyen, Karl Cobbe, Katy Shi, Kayla Wood, Kendra Rimbach, Keren Gu-Lemberg, Kevin Liu, Kevin Lu, Kevin Stone, Kevin Yu, Lama Ahmad, Lauren Yang, Leo Liu, Leon Maksin, Leyton Ho, Liam Fedus, Lilian Weng, Linden Li, Lindsay McCallum, Lindsey Held, Lorenz Kuhn, Lukas Kondraciuk, Lukasz Kaiser, Luke Metz, Madelaine Boyd, Maja Trebacz, Manas Joglekar, Mark Chen, Marko Tintor, Mason Meyer, Matt Jones, Matt Kaufer, Max Schwarzer, Meghan Shah, Mehmet Yatbaz, Melody Y. Guan, Mengyuan Xu, Mengyuan Yan, Mia Glaese, Mianna Chen, Michael Lampe, Michael Malek, Michele Wang, Michelle Fradin, Mike McClay, Mikhail Pavlov, Miles Wang, Mingxuan Wang, Mira Murati, Mo Bavarian, Mostafa Rohaninejad, Nat McAleese, Neil Chowdhury, Neil Chowdhury, Nick Ryder, Nikolas Tezak, Noam Brown, Ofir Nachum, Oleg Boiko, Oleg Murk, Olivia Watkins, Patrick Chao, Paul Ashbourne, Pavel Izmailov, Peter Zhokhov, Rachel Dias, Rahul Arora, Randall Lin, Rapha Gontijo Lopes, Raz Gaon, Reah Miyara, Reimar Leike, Renny Hwang, Rhythm Garg, Robin Brown, Roshan James, Rui Shu, Ryan Cheu, Ryan

Greene, Saachi Jain, Sam Altman, Sam Toizer, Sam Toyer, Samuel Miserendino, Sandhini Agarwal, Santiago Hernandez, Sasha Baker, Scott McKinney, Scottie Yan, Shengjia Zhao, Shengli Hu, Shibani Santurkar, Shraman Ray Chaudhuri, Shuyuan Zhang, Siyuan Fu, Spencer Papay, Steph Lin, Suchir Balaji, Suvansh Sanjeev, Szymon Sidor, Tal Broda, Aidan Clark, Tao Wang, Taylor Gordon, Ted Sanders, Tejal Patwardhan, Thibault Sottiaux, Thomas Degry, Thomas Dimson, Tianhao Zheng, Timur Garipov, Tom Stasi, Trapit Bansal, Trevor Creech, Troy Peterson, Tyna Eloundou, Valerie Qi, Vineet Kosaraju, Vinnie Monaco, Vitchyr Pong, Vlad Fomenko, Weiyi Zheng, Wenda Zhou, Wenting Zhan, Wes McCabe, Wojciech Zaremba, Yann Dubois, Yinghai Lu, Yining Chen, Young Cha, Yu Bai, Yuchen He, Yuchen Zhang, Yunyun Wang, Zheng Shao, and Zhuohan Li. Openai o1 system card, 2026. https://arxiv.org/abs/2412.16720.

Alexandre Piché, Ehsan Kamalloo, Rafael Pardinas, Xiaoyin Chen, and Dzmitry Bahdanau. Pipelinerl: Faster on-policy reinforcement learning for long sequence generation, 2025. https://arxiv.org/abs/2509.19128.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36: 53728–53741, 2023.

Ranajoy Sadhukhan, Zhuoming Chen, Haizhong Zheng, Yang Zhou, Emma Strubell, and Beidi Chen. Kinetics: Rethinking test-time scaling laws, 2025. https://arxiv.org/abs/2506.05333.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025a.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025b.

Hongjin Su, Shizhe Diao, Ximing Lu, Mingjie Liu, Jiacheng Xu, Xin Dong, Yonggan Fu, Peter Belcak, Hanrong Ye, Hongxu Yin, et al. Toolorchestra: Elevating intelligence via efficient model and tool orchestration. *arXiv preprint arXiv:2511.21689*, 2025.

Qidong Su, Christina Giannoula, and Gennady Pekhimenko. The synergy of speculative decoding and batching in serving large language models, 2023. https://arxiv.org/abs/2310.18813.

Hanshi Sun, Li-Wen Chang, Wenlei Bao, Size Zheng, Ningxin Zheng, Xin Liu, Harry Dong, Yuejie Chi, and Beidi Chen. Shadowkv: Kv cache in shadows for high-throughput long-context llm inference. *arXiv preprint arXiv:2410.21465*, 2024a.

Hanshi Sun, Zhuoming Chen, Xinyu Yang, Yuandong Tian, and Beidi Chen. Triforce: Lossless acceleration of long sequence generation with hierarchical speculative decoding, 2024b. https://arxiv.org/abs/2404.11912.

Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context llm inference. *arXiv preprint arXiv:2406.10774*, 2024a.

Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context llm inference, 2024b. https://arxiv.org/abs/2406.10774.

Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.

Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. Trl: Transformer reinforcement learning. https://github.com/huggingface/trl, 2020.

Bo Wu, Sid Wang, Yunhao Tang, Jia Ding, Eryk Helenowski, Liang Tan, Tengyu Xu, Tushar Gowda, Zhengxing Chen, Chen Zhu, Xiaocheng Tang, Yundi Qian, Beibei Zhu, and Rui Hou. Llamarl: A distributed asynchronous reinforcement learning framework for efficient large-scale llm training, 2025a. https://arxiv.org/abs/2505.24034.

Yongji Wu, Xueshen Liu, Haizhong Zheng, Juncheng Gu, Beidi Chen, Z Morley Mao, Arvind Krishnamurthy, and Ion Stoica. Rlboost: Harvesting preemptible resources for cost-efficient reinforcement learning on llms. *arXiv preprint arXiv:2510.19225*, 2025b.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *International Conference on Learning Representations*, 2024. https://openreview.net/forum?id=NG7sS51zVF.

Ran Yan, Youhe Jiang, and Binhang Yuan. Flash sparse attention: More efficient natively trainable sparse attention. *arXiv preprint arXiv:2508.18224*, 2025.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. https://arxiv.org/abs/2505.09388.

Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention, 2025a. https://arxiv.org/abs/2502.11089.

Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Yuxing Wei, Lean Wang, Zhiping Xiao, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 23078–23097, 2025b.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H_2o: Heavy-hitter oracle for efficient generative inference of large language models, 2023. https://arxiv.org/abs/2306.14048.

Haizhong Zheng, Jiawei Zhao, and Bedi Chen. Prosperity before collapse: How far can off-policy rl reach with stale data on llms? *arXiv preprint arXiv:2510.01161*, 2025.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs, 2024. https://arxiv.org/abs/2312.07104.

Yuzhen Zhou, Jiajun Li, Yusheng Su, Gowtham Ramesh, Zilin Zhu, Xiang Long, Chenyang Zhao, Jin Pan, Xiaodong Yu, Ze Wang, Kangrui Du, Jialian Wu, Ximeng Sun, Jiang Liu, Qiaolin Yu, Hao Chen, Zicheng Liu, and Emad Barsoum. April: Active partial rollouts in reinforcement learning to tame long-tail generation, 2025. https://arxiv.org/abs/2509.18521.

Appendix Contents

Limitations

Our study focuses on sparse rollout for RL with verifiable rewards on a limited set of base models, tasks, and hardware settings, so the exact stability thresholds and speedup tradeoffs we report may not transfer unchanged to other model families, reward formulations, or deployment environments. In particular, our conclusions are tied to long-context reasoning workloads and to the sparse-attention configurations we evaluate, and broader validation across additional domains such as multimodal RL, alternative policy optimization algorithms, and more diverse system stacks would be needed before treating the proposed acceptance-rate thresholds and scheduling rules as universal design principles.

Broader Social Impact

If successful, our method can reduce the compute cost of RL training for large language models, which may lower the barrier to studying reasoning systems and make experimentation more accessible to smaller research groups; at the same time, cheaper post-training can also accelerate the development and deployment of increasingly capable models, including systems that may be misused for generating persuasive misinformation, scalable spam, or unsafe automated assistance. We therefore view this work primarily as a systems contribution whose social impact depends on how downstream models are evaluated and released, and we encourage practitioners to pair efficiency gains with careful capability assessment, application-specific safeguards, and responsible release practices.

A Insufficient Rollout Is Not the Main Culprit of Sparse Rollout RL Instability

In this section, we present a concrete study showcasing that although sparse rollout decreases the average RL training reward, the insufficient reward alone does not cause the RL training to crash.

To save computational resources, we don't use the thinking models as we did in the main experiment section, as training on 37K generation length makes rollout extremely cost-heavy. Instead, we base our experiments on top of base models. To make the model CoT generation length longer. We first warmup the standard base models on DeepScaleR dataset Luo et al. (2025) for 20K training examples so that the model average generation length raises to above 6K tokens, from where we then train the model on POLARIS dataset An et al. (2025).

Specifically, for Qwen3-4B-Base model we underwent the above warmup and then start the training with sparse rollout with 12K generation length cutoff. We present RL training trials in Figure 7. We use `pagesize` 16 and `number of pages` 32 for sparse rollout. We compare sparse-rollout RL training (green) with the dense-rollout baseline (purple). On AIME2024 and MATH500, the green curve is consistently below the purple curve. Also note that the sparse-rollout training reward is initially only marginally lower than dense, but eventually crashes.

For sparse rollout, we attempted to increase its average training rewards. In fact, for every training example problem we sample 16 times (instead of N = eight times in green curve), then we rank the 16 rollout trajectories per example problem by their training reward and select the top-8. The average reward curve is shown in blue,

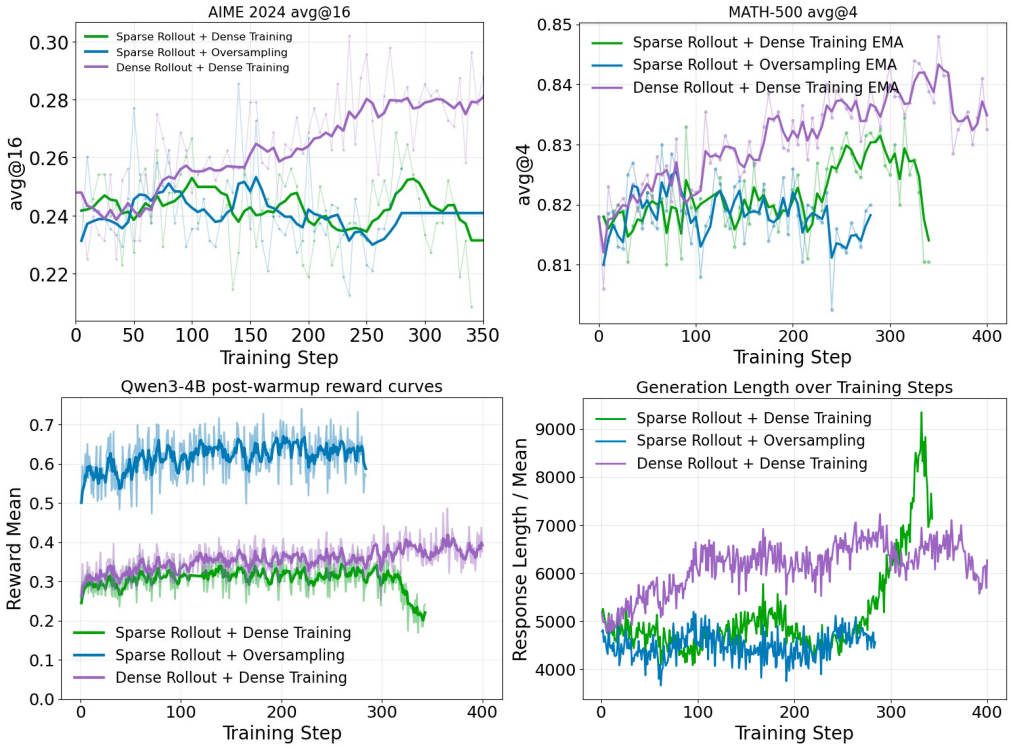

Figure 7 We illustrate with a simple experiment that sparse rollout RL training dense policy collapse is not only due to the reward is insufficient. Oversampling (Sample 16 select Top-8 based on zero-one-reward) drastically increases reward of the rollout but still cannot prevent the training performance to be near the dense rollout.

which is significantly higher than dense average reward and that of the original sparse rollout. However, the blue curve still fails to recover the training performance.

B Extended Related Works

We would like to divide the discussion of the related works into four aspects: RL for LLMs, prior works on distribution alignment in RL, general rollout speedup methods, and sparse attention application in LLM.

RL for LLM. Reinforcement learning has been widely applied to LLMs to improve human alignment, reasoning, coding, and other complex tasks. Beyond PPO, memory efficient methods have been proposed, including ReMax (Li et al., 2023b), RLOO (Ahmadian et al., 2024), and GRPO (Shao et al., 2024). In addition, methods such as SimPO (Meng et al., 2024) and DPO (Rafailov et al., 2023), which are based on offline RL, have also been employed for human alignment. RL training systems for LLMs, such as Verl (Sheng et al., 2025b), AReal (Fu et al., 2025), TRL (von Werra et al., 2020), and OpenRLHF (Hu et al., 2024), have been developed to improve training throughput and scalability.

Distribution Mismatch Correction in RL. Actor-policy mismatch is a common problem that has long been studied, e.g. Impala (Espeholt et al., 2018). To alleviate the actor-policy distribution gap, the method introduces a truncated importance sampling (TIS) to approximate the true PPO objective.

$$\mathcal{L}^{\text{PPO}}(\theta) = \mathbb{E}_{x\sim P_{\text{inf}}}\left[\min\left(\frac{p_{\text{ref}}(x)}{p_{\text{inf}}(x)}, C\right)\min\left(r_\theta(x)\hat{A}(x), \text{clip}(r_\theta(x), 1-\epsilon, 1+\epsilon)\hat{A}(x)\right)\right] \tag{3}$$

The truncation threshold C is for maintaining the stability of the range of the importance ratio. Recently, several methods apply the truncated importance sampling method to RL of LLMs. Methods such as FlashRL (Liu et al., 2025c), AReal (Fu et al., 2025), and LlamaRL (Wu et al., 2025a) address distribution mismatch by introducing (truncated) importance sampling ratios, typically of the form $p_{\text{ref}}/p_{\text{inf}}$, to correct the impact of mismatch on advantage estimation. From system perspective, FP32 LM heads (Liu et al., 2025c) and deterministic LLM Inference (He and Lab, 2025) are implemented to mitigate the numerical issue of serving systems when rollout.

Prior Rollout Speedup Methods. Many recent works have been proposed to address this rollout efficiency challenge, but have several key limitations. Several recent works (Zheng et al., 2025; Piché et al., 2025; Zhou et al., 2025) have designed asynchronous RL training systems that accelerate training by decoupling the rollout and training phases to better utilize computational resources. However, although this line of work prioritizes training throughput, the high resource utilization in the rollout phase remains unaddressed. Moreover, the asynchrony can result in staleness of samples which can potentially lead to inferior training. Another line of work aims to accelerate rollouts with model quantization (Liu et al., 2025c; Huang et al., 2025) and speculative decoding (Leviathan et al., 2023; Chen et al., 2023). Despite that model quantization can significantly reduce the cost of loading model weights, it cannot effectively mitigate the rollout overhead for long-sequence generation, where KV-cache loading remains the primary bottleneck (Sadhukhan et al., 2025). Conversely, speculative decoding can accelerate rollouts without altering the sampling distribution. However, it is largely unsuitable for large-batch rollout setting (Liu et al., 2025d; Su et al., 2023) in RL training because the verification process becomes compute-intensive. Furthermore, speculative decoding introduces an additional draft model which requires extra training resources and thus complicates the whole training pipeline.

Sparse attention. Attention-operation cost dominates the latency of generating long-context output, consensus shared by many prior studies (Sadhukhan et al., 2025; Yuan et al., 2025a). A substantial body of work has focused on reducing the attention cost by pruning redundant token computation and loading and computing only essential tokens. Training-free approaches revolve around fine-grained token-level decisions (Zhang et al., 2023) or more accurate dynamic block-sparse attention (Tang et al., 2024b; Sun et al., 2024b; Liu et al., 2025a). Despite robust performance in general tasks, under aggressive sparsity settings, these methods incur an unacceptable accuracy drop. Pretrained sparse attention methods (Yuan et al., 2025a; DeepSeek-AI, 2025), on the other hand, are achieving scalable results.

C Detailed Implementation and Visualization of the Cost Model

In this section, we provide more details on the cost-model implementation and visualize how `pagesize` relates to cost and distribution mismatch with the dense model. Specifically, for the cost model analysis, we use Qwen3-1.7B model configurations for now. Generalizations to bigger models will be presented in the later versions of the iterations. In particular, we look at `parallel_sampling` to be 8, mimicking the conventional GRPO rollout scenario, and base the analysis on one H200, similar takeaways can also be drawn on B200 as well. (Specifically, we use arithmetic intensity based on H200 but for our conclusion the arithmetic intensity holds when we use B200 arithmetic intensity.) For each `pagesize` setting, we pick 11 KV-budget configurations, covering 0.7 to 0.9 tail mismatch across all experimental sequence lengths.

There are two aspects where we think are worth highlighting. First, `pagesize`=16 occupies the Pareto frontier across all generation lengths. However, as generation length increases, the gap between `pagesize`=32 and 16 shrinks; by 30K tokens, `pagesize`=32 nearly reaches the Pareto frontier and is almost on par with `pagesize`=16 in the cost-mismatch tradeoff. Second, as the generation sequence length increases, the gap between cost of using sparse attention rollout and dense attention increases, showing more cost benefits from using sparse attention.

Table 4 Empirically, we also observe a similar pattern: the page size from 16 and above does not affect the decoding speed a lot as long as the sparse engine's Top-k kernel is well implemented (Cost measured on single H200 with batch size 16 in ms).

Vortex configs (pagesize-num_page)	Past KV Size 16k	24k	32k
166-4	4.59	5.13	5.41
32-32	4.47	4.96	5.30
64-16	4.30	4.87	5.12
128-8	4.13	4.84	5.07

Benefitted from the high-quality top-k kernels supported in Infini-AI-Lab (2026), we are able to empirically show that scoring portion of the sparse attention cost in the model generation is not significantly impacted by the page size selection. Shown in Table 4, we measure the average generation cost in the full forward pass of the sparse attention model under different `pagesize` selections. Particularly, we use a single H200 GPU, and we prefill with sequence lengths 16K, 24K, 32K sequences with batch size 8, and compute the average full-model decoding pass by averaging over 1000 trials. The average is presented in milliseconds (ms).

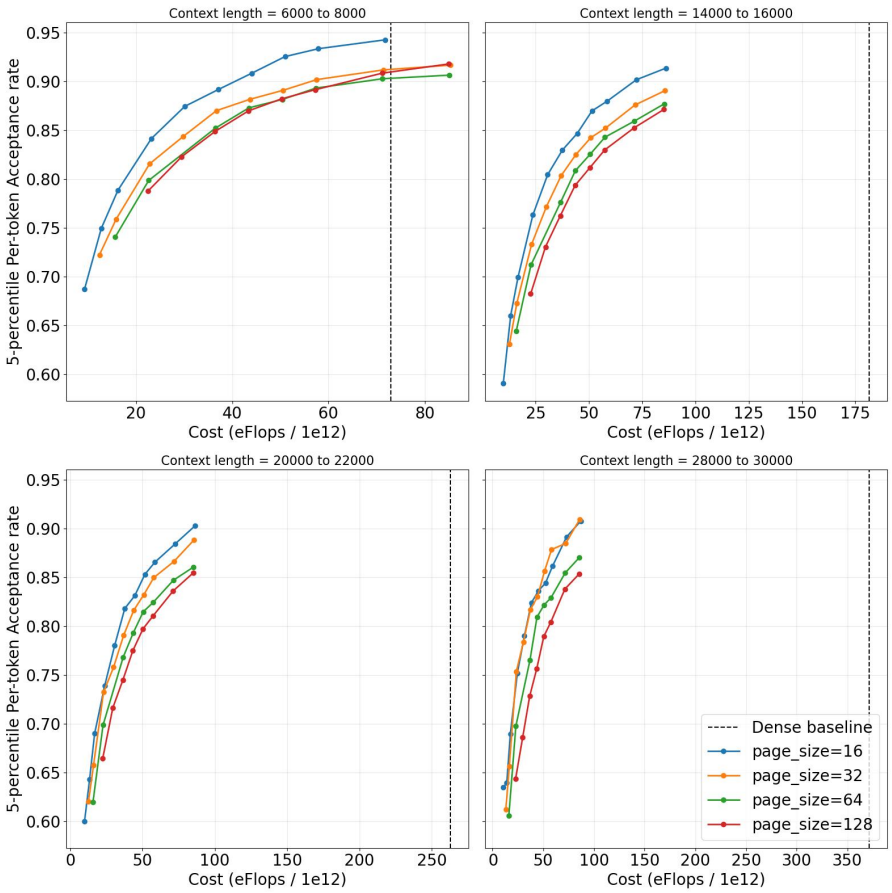

Figure 8 Additional cost analysis showing how different page sizes compare across generation-length regimes.

D Empirical Study of DistillSparse Overhead of LoRA Distillation Training

System implementation & Efficiency: In this section, we show that DistillSparse incurs minimal overhead. To simplify the implementation, for the speedup demonstration, we choose a simplified setting, where instead of using a sparsity schedule for rollout and training with dense distillation, we instead use a fixed sparse budget.

To conduct our experiments, we use Verl (Sheng et al., 2025a) with FSDP as the training engine and SGLang (Zheng et al., 2024) as the inference engine. Experiments are run on Qwen3-4B-Instruct with generation length 16K. Training is run on 2xH200 GPUs. For efficient sparse-attention rollouts, we use Vortex_torch (Chen, 2025). We adopt block top-k attention with a page size of 16, and set the number of top-k pages according to the sparse KV budget. In addition, we use Flash Sparse Attention (Yan et al., 2025) for efficient sparse-attention training and PEFT (Mangrulkar et al., 2022) for LoRA adaptation.

As shown in Figure 9, we report the efficiency of our implementation. When training 4B instruct model with 16K max context length, dense rollouts account for roughly 90% of the per-epoch time. Sparse attention directly alleviates this bottleneck and accelerates rollouts by roughly 1.9×. Although the dense policy update contributes only about 10% of the total cost, our sparse distillation increases this component by approximately 55%. Overall, we obtain a 1.72× end-to-end speedup, showing a promising effect of small overhead.

E Distribution Mismatch under Sparse Rollout Training Collapse: Visualization of KL Divergence of actor-policy mismatch

In this section, we show an illustration of why sparse-dense RL training causes such a huge problem when selecting the sparsity in ad hoc manner. Compared to traditional scenarios such as aggressive staleness where the

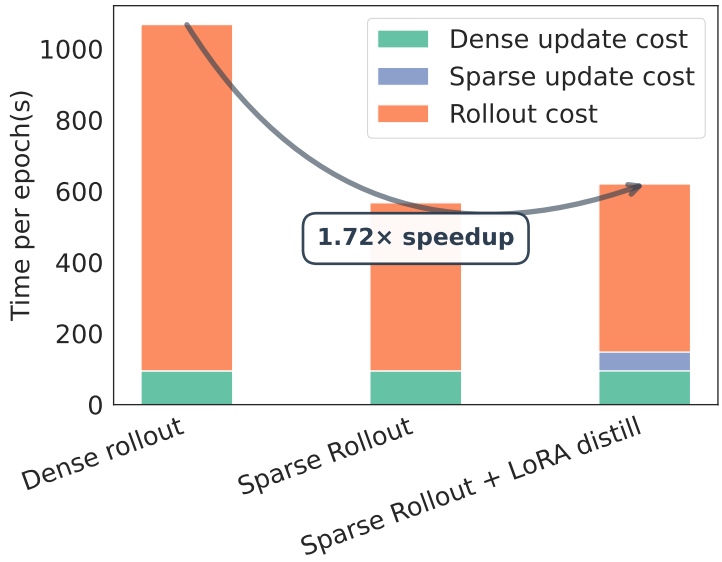

Figure 9

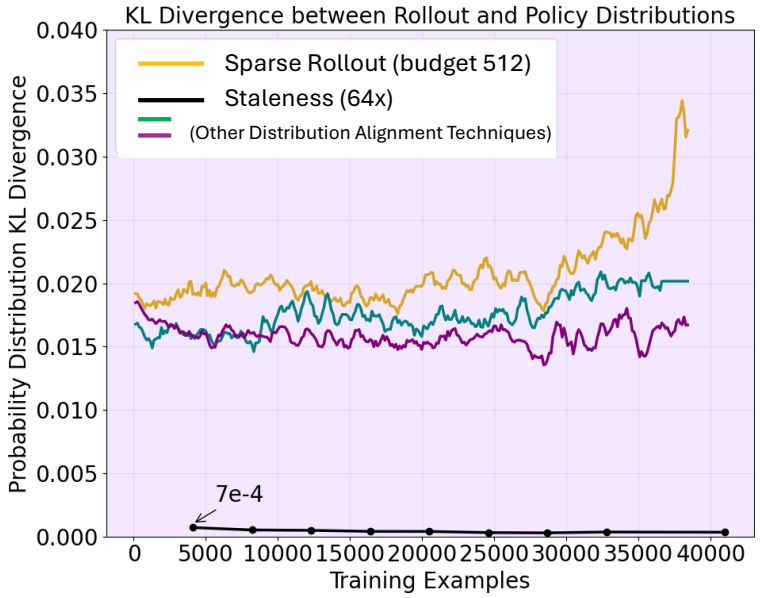

Figure 10 Comparison of actor-policy KL Divergence between actor-policy for Qwen3-4B-Base models under 64 times staleness and sparse to dense training.

distribution alignment techniques like TIS (Liu et al., 2025c) and (Liu et al., 2025b), when choosing aggressive sparsity for rollout, the actor-policy divergence is much more severe, in particular the KL divergence is orders of magnitude higher.

Specifically, we measure the above visualization under the following settings. To save compute, we follow Section A to use Qwen3-4B-Base after warmup with sparse rollout (KV budget 512). With TIS + Jackpot applied, the resulting KL divergence is plotted in the Figure 10 yellow curve, in which it crashes in the later stage of training. In contrast, we also plot the KL divergence of the 64x staleness on Qwen3-4B-Base in black.

F Rollout Cost Analysis and Sparse Attention Motivation

To understand why rollout generation dominates RL training, prior work models decoding cost as the combination of computation and KV-cache memory traffic (Sadhukhan et al., 2025). We adopt the same additive view and analyze rollout latency independently.

For dense autoregressive decoding, the per-layer cost consists of parametric computation plus self-attention:

$$C_{\text{comp}} = 2PBNL_{\text{out}} + rBN(2L_{\text{in}} + L_{\text{out}})L_{\text{out}}D,$$

while memory traffic is driven by parameter and KV-cache access:

$$C_{\text{mem}} = 2PL_{\text{out}} + BN(2L_{\text{in}}L_{\text{out}} + L_{\text{out}}^2)D.$$

Although parameter access can be amortized with large batch size and model parallel inference, the attention term grows quadratically with response length. As a result, rollout decoding becomes increasingly memory-bound for long generations, and self-attention quickly emerges as the dominant bottleneck.

Sparse attention. With a KV budget K, sparse decoding replaces the quadratic dependence on L_{out} with a budgeted cost:

$$C_{\text{comp}}^{(\text{sparse } K)} = 2PBNL_{\text{out}} + rBND \cdot H(L_{\text{in}}, L_{\text{out}}, K),$$
$$C_{\text{mem}}^{(\text{sparse } K)} = 2PL_{\text{out}} + BND \cdot H(L_{\text{in}}, L_{\text{out}}, K),$$

where

$$H = (2L_{\text{in}} + L_{\text{dense}})L_{\text{dense}} + 2K\big(L_{\text{out}} - L_{\text{dense}}\big)_+,$$
$$L_{\text{dense}} = \min(L_{\text{out}}, K - L_{\text{in}}).$$

Thus sparse attention avoids $\mathcal{O}(L_{\text{out}}^2)$ scaling and instead yields $\mathcal{O}(KL_{\text{out}})$ memory growth.

End-to-End Implications: An RL iteration decomposes into rollout generation, logits recomputation, and policy update:

$$C_{\text{total}} = C^{\text{rollout}} + C^{\text{recomp}} + C^{\text{update}}.$$

Recomputation and update operate on full sequences via prefill-style passes, achieving high arithmetic intensity and remaining largely compute-bound. In contrast, rollout decoding performs per-token KV access with low arithmetic intensity, making it fundamentally memory-bound.

Superior Inference Scaling with Sparse Attention. Overall, the rollout phase dominates end-to-end training time (often $> 90\%$, Figure 9), primarily because the KV loading cost increases quadratically with generation length (Sadhukhan et al., 2025). Increasing number of rollouts cannot amortize the decoding cost as the dominant KV memory also grows linearly with it. Sparse attention directly targets this bottleneck by reducing rollout memory cost from $\mathcal{O}(L_{\text{out}}^2)$ to $\mathcal{O}(KL_{\text{out}})$, where K is the sparse KV cache size. This allows us to increase the *generation length* and *number of samples* to achieve high-quality rollouts within reasonable cost. In other words, **sparse attention unlocks superior inference scaling in the usual long CoT regime.**